A PROJECT ON

SMART AI-POWERED DOOR LOCK SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by

NAME-Yugank Prajapati **University Roll No-2219986**

NAME- Gagandeep Singh **University RollNo- 2218745**

NAME-Nimansi Dixit **University Roll No-2219170**

Under the guidance of

Ms. Manisha Aeri

(Assistant Professor)

DESIGNATION

Project Group No:532

Department of Computer Science and Engineering

Graphic Era Hill University

May, 2026

Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the project progress report entitled **"Smart AI-Powered Door Lock System"** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering in the Department of Computer Science and Engineering of the Graphic Era Hill University, Dehradun shall be carried out by the undersigned under the supervision of **Ms. Manisha Aeri (Assistant Professor),** Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun.

NAME- Yugank Prajapati University Roll No- 2219986

NAME- Gagandeep Singh University Roll No- 2218745

NAME: Nimansi Dixit University Roll No- 2219170

The students mentioned above shall be working under the supervision of the undersigned on the **"Smart AI-Powered Door Lock System"**

Guide - **Ms. Manisha Aeri**

Head of the Department - **Prof. Anupam Singh**,

Date -

Place - **Dehradun, Uttrakhand**

ACKNOWLEDGEMENT

Any achievement, be it scholastic or otherwise, does not depend solely on individual effort but on the guidance, encouragement, and cooperation of intellectuals, elders, and friends.

Several personalities in their capacity have helped me in carrying out this project work. Our sincere thanks to project guide **Ms. Mansiha Aeri Mam,** Assistant Professor, Dept. of CSE, Graphic Era (Deemed to be University), for his valuable guidance and support throughout the course of project work and for being a constant source of inspiration.

We are extremely grateful to **Prof. Anupam Singh**, HOD of the Computer Science and Engineering Department, Graphic Era hill to be University, for his moral support and encouragement.

We thank the **management of Graphic Era (Deemed to be University)** for the support throughout our Bachelor's Degree and for all the facilities they have provided.

Name of student – Yugank Prajapati
Roll no- 2219986

Name of student- Gagandeep Singh
Roll no-2218745

Name of student- Nimansi Dixit
Roll no-2219170

ABSTRACT

In recent years, smart security systems have gained significant importance due to the increasing need for reliable and automated access control. This project presents the design and development of a Smart AI-Powered Door Lock System based on face recognition technology, enabling secure and contactless authentication for authorized visitors.

Face verification is achieved using the DeepFace framework with the FaceNet512 model for feature extraction and comparison. During enrollment, visitor images are captured and stored in the system database as reference data. At runtime, a live image is captured through a camera interface, processed, and matched against the stored dataset using cosine similarity metrics. If the similarity score satisfies a predefined threshold, the visitor is successfully verified and granted access.

The system supports efficient visitor management through a web-based dashboard, allowing owners to register, update, and monitor visitor data. Additionally, a desktop-based graphical user interface (GUI) enables real-time face verification using live camera feeds, ensuring seamless on-site authentication. The system provides instant feedback on verification results, improving usability and reliability.

The entire solution follows a modular architecture comprising a React-based frontend for user interaction, a Node.js and Express backend for API handling and authentication, and Python microservices integrating DeepFace and OpenCV for AI-driven processing. Secure communication is ensured using JWT-based authentication, HTTP-only cookies, and robust data validation mechanisms.

Future enhancements include integration with IoT-based smart locks, mobile application support, advanced activity logging, multi-factor authentication, and cloud deployment. By combining artificial intelligence with modern web technologies, this system offers a scalable and efficient approach to smart access control in residential, academic, and commercial environments.

TABLE OF CONTENT

ACKNOWLEDGEMENT	**3**
ABSTRACT	**4**
LIST OF TABLES	**7**
LIST OF FIGURES	**8**
Chapter 1: Introduction and Motivation	**9 – 21**
1.1 Introduction	9 – 10
1.2 Project Overview	10 – 11
1.3 Project Objectives	11 – 14
1.4 Scope of the Project	15 – 17
1.5 Benefits	18 – 20
1.6 Use Cases	20 – 21
Chapter 2: Literature Survey / Background	**22 –28**
2.1 Face Recognition and Biometric Security	23 – 24
2.2 Deep Learning for Face Recognition	25 – 25
2.3 Smart Security Systems and Access Control	26 – 26
2.4 Literature Survey	27 - 28
Chapter 3: Software Design and Working	**29 - 50**
3.1 Data Flow Diagram	29 – 32
3.2 System Architecture Overview	33 – 37
3.3 System Components Overview	37 – 37
3.4 Requirements	38 – 40
3.5 Working	41 – 45
3.6 User Interface	45 - 50
Chapter 4: Results	**51 - 54**
4.1 Face Recognition Performance	51 – 51
4.2 Visitor Authentication and Access Control	51 – 52
4.3 System Functionality and Execution	52 – 52
4.4 Security and System Authentication	52 – 53
4.5 Test Results and Accuracy	53 – 53
4.6 Overall System Evaluation	54 - 54

Chapter 5: Conclusion and Future Scope **55 - 58**
5.1 Conclusion 55 – 56
5.2 Future Work 57 - 58

Chapter 6: Coding **59 – 67**
6.1 Backend (Node.js + Express API) 59 – 61
6.2 Frontend (React + Tailwind Dashboard) 62 – 63
6.3 Python Microservice (Deepface + Flask) 64 – 64
6.4 Device GUI (Tkinter + OpenCV) 65 - 65

REFERENCES **66 - 67**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	System Components Overview	37
3.2	Deepface Working	44
4.1	Test Results	53

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Smart AI-Powered Door Lock System Overview	10
2.1	Face Recognition Process Flow	24
2.2	Face Recognition using Deepface	28
3.1	Data Flow diagram	29
3.2	System Architecture Overview	33
3.3	Data Flow Diagram (Level 1)	40
3.4	Working	41
3.5	Dashboard	46
3.6	Visitor Management	47
3.7	Activity Log	47
3.8	Access Granted	49
3.9	Access Denied	49
6.1	auth.js	59
6.2	visitor.js	60
6.3	verify.js	61
6.4	dashboard.js	62
6.5	api.js	63
6.6	deepface_service.py	64
6.7	gui.py	65

CHAPTER 1

INTRODUCTION AND MOTIVATION

1.1. INTRODUCTION

In today's rapidly evolving digital era, security and access control have become critical concerns in residential, commercial, and institutional environments. Traditional locking mechanisms such as keys, PINs, and access cards are increasingly becoming inadequate due to risks like duplication, theft, and unauthorized sharing. As a result, there is a growing demand for intelligent, automated, and contactless security systems that provide both convenience and enhanced protection.

With the advancement of Artificial Intelligence (AI) and computer vision technologies, biometric authentication has emerged as a reliable solution for secure access control. Among various biometric techniques, face recognition has gained significant popularity due to its non-intrusive nature and ability to authenticate users without physical contact. It enables systems to identify individuals based on unique facial features, making it highly suitable for real-time applications.

To address these challenges, the **Smart AI Powered Door Lock System** is developed as an intelligent access control solution that integrates face recognition with modern web technologies. The system is designed to allow only authorized individuals to gain access by verifying their identity through facial recognition. Unlike traditional systems, it eliminates the need for physical keys or manual intervention, thereby enhancing both security and user convenience.

The project combines a web-based dashboard for managing users and visitors, a backend server for handling authentication and data processing, and Python-based microservices for performing AI-driven face recognition. A desktop graphical interface enables real-time verification using live camera input, ensuring accurate and efficient identification of visitors.

Security is a core aspect of the system. It incorporates mechanisms such as JWT-based authentication, secure data handling, and controlled access to system resources. Additionally, all verification activities are processed in real-time, providing immediate feedback on access decisions.

This project demonstrates how AI and modern software architecture can be integrated to build a scalable, secure, and user-friendly smart security system. It lays the foundation for future advancements such as IoT-based smart locks, cloud deployment, and enhanced biometric authentication systems.

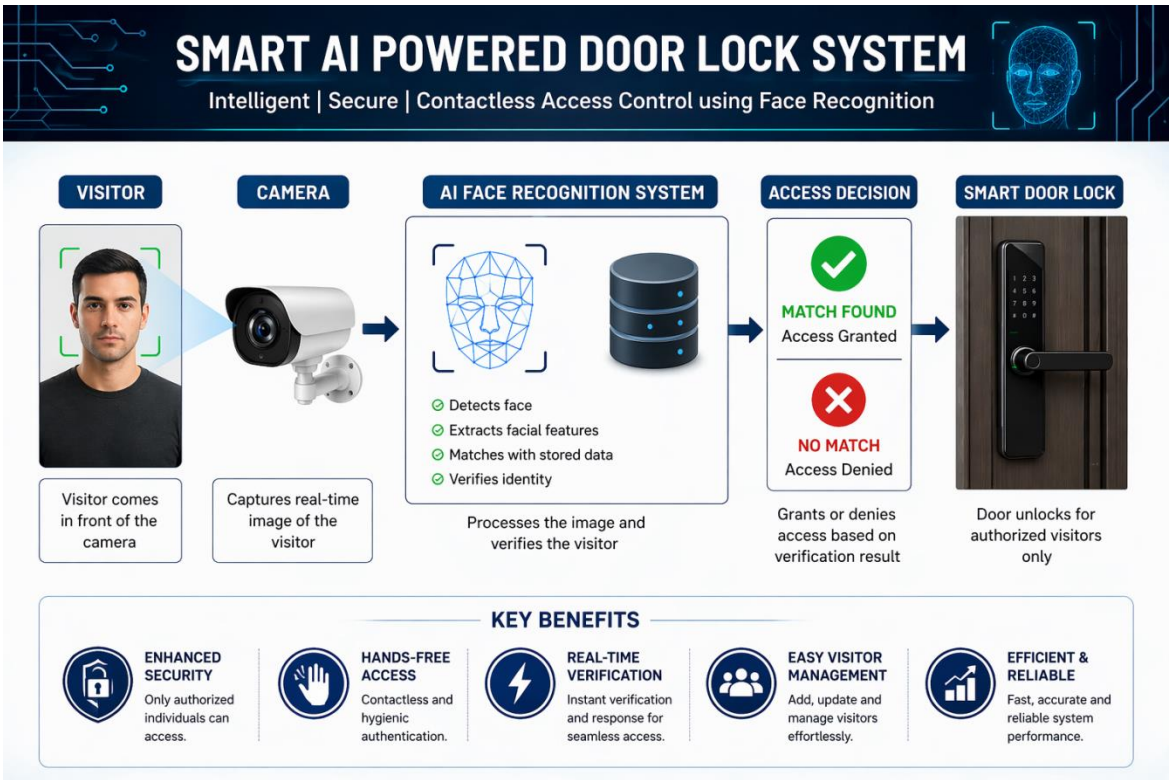


Figure 1.1 Smart-AI Powered Door Lock System Overview

1.2. PROJECT OVERVIEW

The Smart AI Powered Door Lock System is a real-time, AI-based security solution designed to provide secure and automated access control using face recognition technology. The primary objective of the system is to ensure that only authorized visitors are granted access while maintaining ease of use for system owners.

The system operates through three major components: a web dashboard, a backend server, and a face recognition module. The web dashboard allows the owner to register and manage visitor details, including storing their facial images. These images are used as reference data for future verification.

The backend, developed using Node.js and Express, handles authentication, API communication, and data management. It acts as a bridge between the frontend interface and the AI-based face recognition service. Secure communication is maintained using JSON Web Tokens (JWT) and HTTP-only cookies.

The face recognition functionality is implemented using Python microservices powered by the DeepFace framework and FaceNet512 model. During verification, a live image is captured using a camera through a desktop GUI application. The system then compares the captured image with stored visitor images using cosine similarity metrics. If the similarity score meets the defined threshold, the visitor is verified successfully.

The system provides real-time feedback through the GUI, displaying whether access is granted or denied. This ensures transparency and improves user experience. Additionally, the modular design of the system allows easy integration of new features and scalability for future enhancements. Overall, the project presents a practical implementation of AI-based security that combines accuracy, efficiency, and usability.

1.3. PROJECT OBJECTIVES

1.3.1. Secure Face-Based User Authentication

The foremost objective of this project is to implement a secure and reliable authentication mechanism using facial recognition technology. Unlike traditional methods such as passwords, PINs, or physical keys, which can be easily lost, stolen, or duplicated, face recognition provides a biometric-based solution that is unique to each individual.

In this system, every authorized visitor is registered with one or more facial images, which are stored in the database as reference data. These images serve as biometric identifiers and are used during the verification process. When a visitor attempts to gain access, the system captures a live image and compares it with the stored dataset using advanced facial recognition techniques.

The use of the DeepFace API ensures that the authentication process is both accurate and efficient. By leveraging pre-trained deep learning models, the system is capable of identifying subtle differences in facial features, making it highly secure against unauthorized access.

This objective ensures that the system eliminates dependency on traditional authentication methods and replaces them with a more secure, contactless, and intelligent solution.

1.3.2. High-Accuracy Face Recognition using AI

Another critical objective of the project is to achieve high accuracy in face recognition by utilizing artificial intelligence and deep learning techniques. The system uses advanced models such as FaceNet512 through the DeepFace framework to extract facial embeddings and perform comparison.

Facial recognition systems often face challenges such as variations in lighting conditions, facial expressions, camera angles, and partial occlusions. To address these challenges, the system relies on deep learning models that are trained on large-scale datasets and capable of generalizing across different scenarios.

The extracted facial embeddings represent unique numerical features of a person's face, which are then compared using similarity metrics such as cosine distance. This approach ensures that even minor variations in appearance do not significantly affect the recognition accuracy.

The goal is to maintain a high level of precision and reliability in real-time environments, making the system suitable for practical deployment in homes, offices, and institutional settings.

1.3.3. Real-Time Visitor Verification

The system is designed to perform real-time verification of visitors using a live camera feed. This objective focuses on minimizing delays in the authentication process and providing instant access decisions.

When a visitor arrives, the system captures an image through a camera interface integrated with the desktop GUI. This image is immediately processed by the backend and sent to the DeepFace module for verification. The entire process—from image capture to access decision—is completed within a few seconds.

Real-time verification is essential for maintaining a smooth user experience, especially in environments where quick access is required. It also enhances security by ensuring that verification decisions are made instantly without manual intervention.

By achieving real-time processing, the system ensures efficiency, responsiveness, and practicality in real-world applications.

1.3.4. Web-based Visitor Management System

A key objective of the project is to provide a comprehensive web-based dashboard for managing visitor data. The system includes a frontend interface developed using React, which allows users to interact with the system in an intuitive manner.

Through the dashboard, the system owner can:

- Register new visitors
- Upload and manage facial images
- Update visitor information
- Delete outdated records
- Monitor system activity

This centralized management system ensures that all visitor-related operations can be performed efficiently without requiring technical expertise. The use of a web-based interface also allows remote access, enabling users to manage the system from any location.

The backend, developed using Node.js and Express, handles all API requests and ensures secure communication between the frontend and the database. This objective enhances usability and makes the system practical for everyday use.

1.3.5. Secure System Architecture

Security is a fundamental objective of this project. The system is designed with multiple layers of security to protect user data and prevent unauthorized access.

Key security features include:

- JWT (JSON Web Token) based authentication
- Secure password hashing using encryption algorithms
- Protected API routes
- Input validation and error handling
- Secure storage of user and visitor data

The system ensures that only authenticated users can access the dashboard and perform operations. Additionally, the face recognition module acts as an extra layer of biometric security, making the system highly robust.

1.3.6. User-Friendly Interface

Another important objective is to design an intuitive and user-friendly interface that can be easily used by individuals with varying levels of technical expertise.
The system provides two main interfaces:

- A web-based dashboard for management
- A desktop GUI for real-time verification

The web interface is designed using modern frontend technologies to ensure responsiveness and ease of navigation. The layout is clean, organized, and visually appealing, allowing users to perform tasks efficiently.

The desktop GUI, developed using Tkinter and OpenCV, provides a simple interface for capturing images and displaying verification results. Clear messages such as "Access Granted" or "Access Denied" are shown to ensure transparency.
This objective ensures that the system is not only technically advanced but also accessible and easy to use.

1.3.7. Scalability and Future Expansion

The final objective of the project is to design a system that is scalable and capable of supporting future enhancements. The modular architecture of the system allows easy integration of new features without affecting existing functionality.
Potential future expansions include:

- Integration with IoT-based smart locks
- Development of mobile applications
- Implementation of multi-factor authentication
- Cloud deployment for centralized access
- Addition of real-time notifications

The use of modern technologies such as MERN stack and microservices architecture ensures that the system can be extended to support large-scale deployments.
This objective ensures that the project is not limited to its current implementation but can evolve into a more advanced and comprehensive smart security solution.

1.4. Scope of the Project

1.4.1. Face Recognition-Based Access Control

The primary scope of the system lies in implementing a face recognition-based access control mechanism that replaces traditional authentication methods such as keys, passwords, and access cards.

In this system, access is granted only after verifying the identity of a visitor using facial recognition. Each registered visitor has associated facial images stored in the database, which act as biometric identifiers. During verification, a live image is captured and compared with stored data using the DeepFace API.

This approach ensures:

- Elimination of physical keys and manual authentication
- Reduced risk of unauthorized access
- Contactless and hygienic operation
- Improved convenience and automation

The system is designed to function effectively in controlled environments such as homes, offices, and institutions, where secure entry management is required.

1.4.2. Real-Time Image Processing

Another important aspect of the project scope is real-time image capture and processing. The system is capable of capturing a visitor's image through a camera interface and processing it instantly for verification.

The captured image is sent to the backend and further to the AI microservice, where facial detection, preprocessing, and feature extraction are performed. The system generates facial embeddings and compares them with stored data in real time.

The entire process is optimized to:

- Minimize latency
- Provide quick access decisions (within seconds)
- Maintain high accuracy even under varying conditions

Real-time processing ensures that the system can be used in practical scenarios where immediate authentication is required without delays.

1.4.3. Visitor Data Management

The system includes a centralized data management module for handling visitor information. This module is implemented through a web-based dashboard that allows users to efficiently manage all records.

The scope of this component includes:

- Storing visitor details such as name and associated images
- Updating and modifying existing records
- Deleting outdated or unnecessary entries
- Maintaining a structured database for quick retrieval

The backend database (MongoDB) ensures efficient storage and retrieval of data, while the API layer handles communication between the frontend and database.

Proper data management is essential for maintaining the accuracy and reliability of the face recognition system, as the quality of stored data directly impacts verification performance.

1.4.4. Multi-Component System Integration

The project is designed as a multi-component system that integrates various technologies into a unified architecture. This integration is a key part of the project scope.The system consists of:

- A frontend (React-based web dashboard)
- A backend (Node.js + Express API)
- A database (MongoDB)
- AI microservices (Python + DeepFace)
- A desktop GUI (Tkinter + OpenCV)

Each component performs a specific role, and they communicate through well-defined APIs and data flows.

This modular integration ensures:

- Separation of concerns
- Easier maintenance and debugging
- Flexibility in upgrading individual components
- Efficient data flow across the system

The integration of multiple technologies demonstrates the system’s capability to handle complex operations in a coordinated manner.

1.4.5. Desktop-Based Verification System

The system includes a desktop-based graphical user interface (GUI) for performing on-site verification. This component is responsible for interacting directly with the camera and capturing live images of visitors.

The GUI is developed using Tkinter and integrated with OpenCV for handling camera operations. It provides:

- Live camera feed display
- Image capture functionality
- Real-time verification result display

The desktop interface ensures that the system can operate independently at the entry point, making it suitable for real-world deployment.

This component enhances usability by providing a simple and focused interface for verification, without requiring users to interact with complex backend systems.

1.4.6. Future Integration and Scalability

The system is designed with scalability and future expansion in mind. The modular architecture allows easy integration of additional features and technologies.

The scope for future enhancements includes:

- Integration with IoT-based smart locks for automatic door control
- Deployment on cloud platforms for centralized access and scalability
- Development of mobile applications for remote monitoring and control
- Implementation of multi-factor authentication for enhanced security
- Addition of real-time notifications and activity tracking

The use of modern technologies ensures that the system can evolve with changing requirements and be adapted for larger-scale deployments.

1.5. BENEFITS

1.5.1. Improved Security

One of the most significant benefits of the system is enhanced security through biometric authentication. Unlike traditional methods such as keys, passwords, or access cards, facial recognition relies on unique biological characteristics that are difficult to replicate or forge.

The system uses the DeepFace API to perform accurate face verification using pre-trained deep learning models. This ensures that only authorized individuals can gain access to the system.

Additionally, the system incorporates multiple layers of security, including:

- JWT-based user authentication
- Secure password hashing
- Protected API routes

These combined measures significantly reduce the risk of unauthorized access, making the system highly reliable for secure environments.

1.5.2. Hands-Free Interaction

The system provides a completely contactless authentication process, allowing users to gain access without touching any device. This is particularly beneficial in maintaining hygiene and reducing the spread of germs, especially in public or shared environments.

Unlike fingerprint scanners or keypad systems, face recognition eliminates the need for physical interaction. Users simply stand in front of the camera, and the system automatically performs verification.

This hands-free approach improves convenience and enhances the overall user experience, making the system suitable for modern smart environments.

1.5.3. Real-Time Access Control

The system is designed to provide real-time access control by performing instant verification of visitors. The integration of camera-based image capture and AI-based processing enables quick decision-making.

The entire process—from image capture to verification—is completed within a few seconds. This ensures:

- Minimal waiting time
- Smooth entry process
- Efficient handling of multiple users

Real-time response is critical in environments where quick access is required, such as offices or residential buildings.

1.5.4. Easy Management

The system includes a web-based dashboard that simplifies the management of visitors and system operations. The interface is designed to be intuitive and easy to use, allowing even non-technical users to operate the system efficiently.

Through the dashboard, users can:

- Add new visitors
- Upload and manage facial images
- Update or delete records
- Monitor system activity

This centralized management system eliminates the need for manual record-keeping and improves overall efficiency.

1.5.5. Scalable and Flexible Design

The system is built using a modular architecture that supports scalability and flexibility. Each component—frontend, backend, database, and AI module—is designed to function independently while maintaining seamless integration.

This allows:

- Easy addition of new features
- Integration with external systems
- Expansion to support multiple users or locations

The use of modern technologies ensures that the system can evolve with future requirements without major modifications.

1.5.6. Reduced Dependency on Physical Keys

Traditional access control systems rely heavily on physical keys, which can be lost, stolen, or duplicated. This creates significant security risks and inconvenience.

The proposed system eliminates the need for physical keys by using facial recognition as the primary authentication method. This reduces:

- Risk of unauthorized duplication
- Dependency on manual key management
- Cost associated with replacing lost keys

By removing reliance on physical credentials, the system provides a more secure and efficient solution.

1.6. USE CASES

1.6.1. Smart Homes

In residential settings, the system provides secure and automated entry for homeowners and registered visitors. It eliminates the need for keys and ensures that only authorized individuals can access the premises.

The system can also be integrated with smart home devices, enabling automation such as unlocking doors upon successful verification.

1.6.2. Office and Workspaces

In corporate environments, the system can be used to control access to office premises and restricted areas. It ensures that only employees or authorized personnel can enter specific zones.

This improves workplace security and reduces the need for manual access control systems such as ID cards or security personnel..

1.6.3. Educational Institution

The system can be deployed in schools, colleges, and universities for secure access to facilities such as laboratories, libraries, and classrooms.

It can also be extended to track attendance and monitor entry of students and staff, improving overall campus security.

1.6.4. Hostels and Apartments

In residential complexes, hostels, and apartments, the system can manage visitor access efficiently. It ensures that only registered individuals are allowed entry, enhancing safety for residents.
The system also simplifies visitor management for security personnel.

1.6.5. Healthcare Facilities

In hospitals and healthcare centers, the system can be used to restrict access to sensitive areas such as patient wards, laboratories, and operation rooms.
This ensures that only authorized medical staff can access critical zones, maintaining safety and privacy.

1.6.6. Commercial Buildings

In commercial environments such as malls, hotels, and corporate buildings, the system enhances security and monitoring.
It can be used to control entry into restricted areas, track visitor movement, and improve overall management of large-scale facilities.

CHAPTER 2

Literature Survey/ Background

With the advancement of artificial intelligence and computer vision, face recognition has become a key technology in modern security systems. Traditional methods such as keys, passwords, and access cards are prone to loss and unauthorized access, creating the need for more secure and reliable authentication techniques.

Biometric authentication, particularly face recognition, offers a contactless and user-friendly solution by identifying individuals based on unique facial features. Face recognition systems mainly perform two tasks: identification and verification, where verification ensures that a person matches a registered identity.

Deep learning models such as Convolutional Neural Networks (CNNs) and FaceNet have significantly improved the accuracy and efficiency of face recognition systems. These models extract facial embeddings and compare them using similarity measures to enable real-time authentication.

By integrating these technologies, the proposed system provides a secure and intelligent door lock solution using face recognition, supported by a web-based dashboard, backend API, and real-time verification system.

2.1. FACE RECOGNITION AND BIOMETRIC SECURITY

Biometric authentication has become a key component in modern security systems due to its ability to uniquely identify individuals based on physiological or behavioral traits. Unlike traditional authentication methods such as passwords or PINs, biometric systems provide higher security and convenience.

Biometric systems are broadly classified into physiological (face, fingerprint, iris) and behavioral (voice, gait, typing patterns). Among these, face recognition has gained widespread adoption due to its contactless nature and ease of use in real-time applications.

2.1.1. What is Biometric Authentication?

Biometric authentication is a security process that verifies the identity of an individual using unique biological or behavioral characteristics. These characteristics are difficult to duplicate, making biometric systems more reliable than traditional authentication methods.

Common types of biometric authentication include:

1. Fingerprint Recognition
2. Face Recognition
3. Iris/Retina Scanning
4. Voice Recognition
5. Hand Geometry
6. Behavioral Biometrics

Face recognition systems capture facial images, extract key features such as distances between facial landmarks, and generate a unique representation. This representation is compared with stored data to verify identity.

2.1.2. Face Recognition for Authentication

Face recognition is a biometric technique that identifies or verifies a person based on facial features. It works by analyzing patterns such as facial geometry, texture, and spatial relationships between facial components.

Modern face recognition systems use deep learning models to extract embeddings (numerical representations) of faces. These embeddings are then compared using similarity measures to determine identity.

Face recognition is widely used in:

- Smart security systems
- Surveillance systems
- Attendance systems
- Mobile authentication

In this project, face recognition is used to authenticate visitors before granting access to the door lock system, ensuring secure and contactless entry.

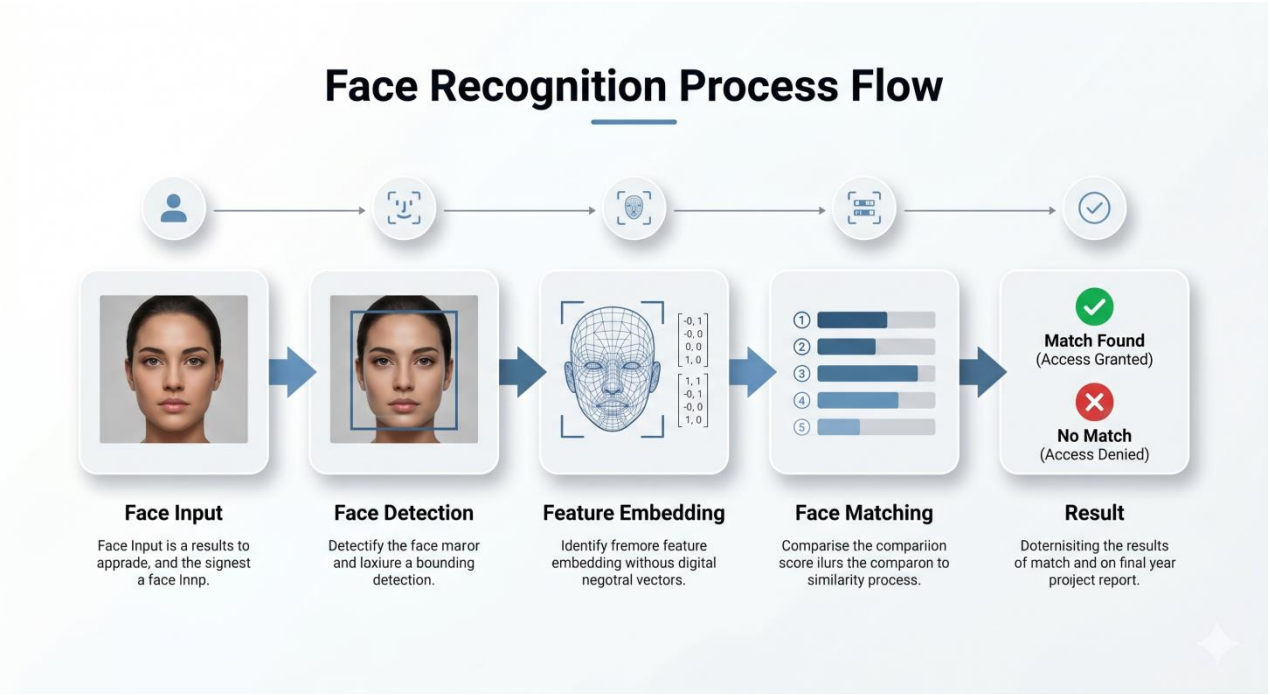


Figure 2.1 Face Recognition Process Flow

2.2. DEEP LEARNING FOR FACE RECOGNITION

Deep learning has significantly improved the performance of face recognition systems. Traditional methods relied on manual feature extraction techniques, which were limited in handling real-world variations.

Deep learning models, especially Convolutional Neural Networks (CNNs), automatically learn important features from images, leading to higher accuracy and robustness.

2.2.1. Why Deep Learning?

Deep learning provides several advantages over traditional methods:

- *Automatic Feature Extraction*: No need for manual feature engineering
- *High Accuracy*: Better performance in complex environments
- *Robustness*: Handles variations in lighting, pose, and expressions
- *Scalability*: Supports large datasets and multiple users
- *Real-Time Processing*: Fast inference for live applications

These advantages make deep learning suitable for real-time face recognition systems.

2.2.2. Face Recognition Using Deep Learning Models

Deep learning models such as CNNs process facial images to extract meaningful features. These features are converted into embeddings, which uniquely represent each face.

Frameworks like DeepFace integrate advanced models such as FaceNet, which generate high-quality embeddings. These embeddings are compared using cosine similarity to verify identity.

In this project, the DeepFace framework with the FaceNet512 model is used. The system captures a live image, extracts facial features, and compares them with stored visitor data to determine whether access should be granted.

2.3 Smart Security Systems and Access Control

Smart security systems combine modern technologies such as AI, IoT, and cloud computing to provide advanced access control solutions. These systems improve security while offering automation and remote management capabilities.

2.3.1 Traditional vs Smart Security Systems

Traditional systems rely on:

- Physical keys
- Passwords or PINs
- Access cards

These methods have limitations such as theft, duplication, and unauthorized sharing.

Smart security systems, on the other hand, use:

- Biometric authentication
- Remote access control
- Real-time monitoring
- Cloud-based management

These features make them more secure and efficient.

2.3.2 Face Recognition-Based Access Control

Face recognition-based systems provide a contactless and secure method for access control. They eliminate the need for physical credentials and reduce the risk of unauthorized access.

- Key features include:
- Automatic identity verification
- Real-time access decisions
- Integration with web and mobile applications
- Activity monitoring and logging

2.4 Literature Survey

The field of face recognition and smart security systems has been extensively studied, with continuous improvements in accuracy and efficiency.

2.4.1 Traditional Approaches

Earlier face recognition systems used techniques such as:

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)

These methods were limited in handling variations in lighting, pose, and facial expressions.

2.4.2 Deep Learning-Based Approaches

With the advancement of deep learning, researchers introduced CNN-based models for face recognition. Models such as FaceNet and VGG-Face achieved high accuracy by learning complex facial features automatically.

These models significantly improved performance in real-world conditions and enabled real-time applications.

2.4.3 Smart Security System Implementations

Recent studies focus on integrating face recognition with access control systems. These systems combine:

- Image capture devices
- AI-based recognition models
- Backend servers for data management
- User interfaces for control and monitoring

Such systems are widely used in smart homes, offices, and public security applications.

2.4.4 Proposed System Contribution

The proposed Smart AI Powered Door Lock System builds upon these advancements by integrating:

- Deep learning-based face recognition (DeepFace + FaceNet512)
- MERN stack for web-based management
- Python microservices for AI processing
- Desktop GUI for real-time verification

This combination provides a complete, scalable, and efficient access control solution suitable for real-world deployment.

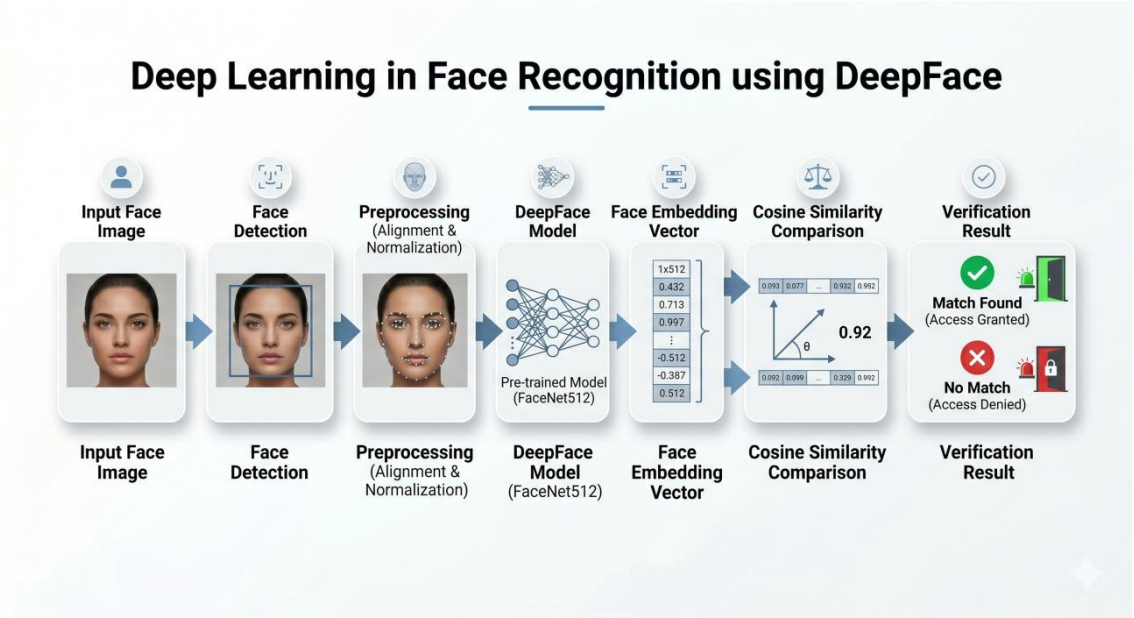


Figure 2.2 Face Recognition using Deepface

CHAPTER 3

SOFTWARE DESIGN AND WORKING

3.1. DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) represents how data moves through the Smart AI Powered Door Lock System. It shows the interaction between the user, web dashboard, backend server, and face recognition system**.**

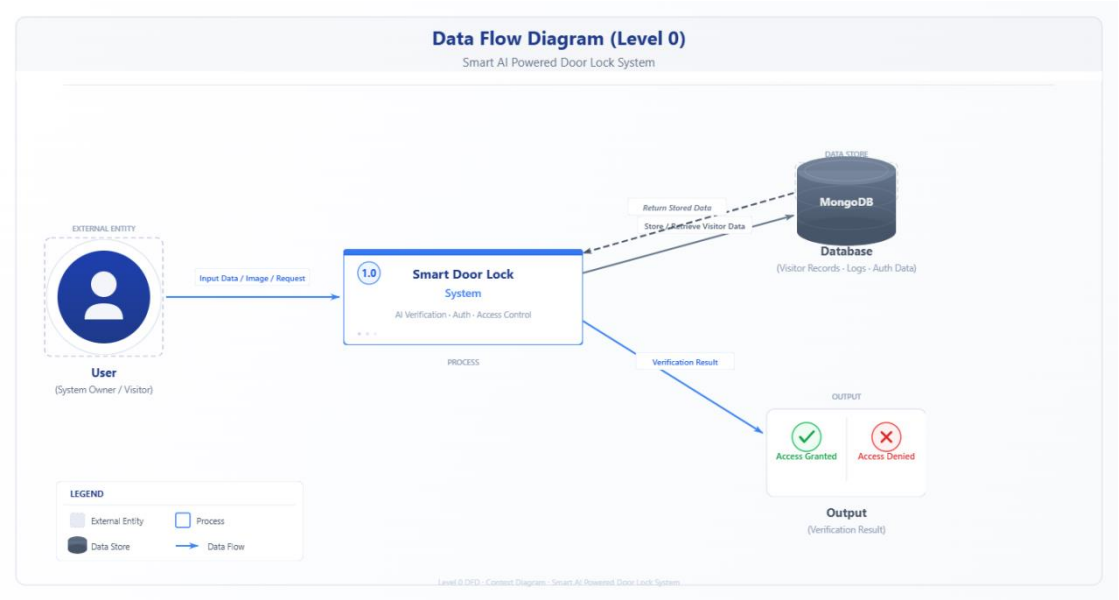


Figure 3.1 Data Flow diagram

3.1.1. Web Interface (Frontend)

The web interface serves as the primary interaction layer between the system owner and the Smart AI Powered Door Lock System. It is developed using modern frontend technologies such as React, Tailwind CSS, and JavaScript, ensuring a responsive, dynamic, and user-friendly experience.

The frontend is designed following component-based architecture, where each functionality is divided into reusable components. This approach improves maintainability, scalability, and performance of the application.

Through this interface, the system owner can perform various operations, including:

- Registering and logging into the system securely
- Adding new visitors along with their facial images
- Viewing, updating, and deleting visitor records
- Monitoring system activity and verification logs

The interface is visually structured with clear navigation and intuitive controls, allowing users to perform actions efficiently without requiring technical expertise.

All user interactions on the frontend are converted into API requests and sent to the backend server using HTTP protocols. These requests include data such as user credentials, visitor information, and image uploads. The frontend communicates with the backend through RESTful APIs, ensuring seamless data exchange.

Additionally, the use of Tailwind CSS ensures a consistent and responsive design across different devices and screen sizes. The frontend also handles input validation and basic error handling before sending data to the backend, improving system reliability.

3.1.2. Backend Server (Node.js + Express)

The backend server acts as the central processing unit of the system, coordinating all operations and managing communication between different components. It is implemented using Node.js and Express, which provide a lightweight and efficient environment for building scalable web applications.

The backend is responsible for handling core functionalities such as:

- User authentication and authorization using JWT (JSON Web Tokens)
- Processing API requests received from the frontend
- Performing database operations such as storing and retrieving data
- Communicating with the Python-based face recognition microservice
- Managing application logic and enforcing security policies

When a user performs an action on the frontend, the request is sent to the backend API. The backend validates the request, processes the data, and performs the required operation. For example, when a new visitor is added, the backend stores the visitor's details and image references in the database. JWT-based authentication ensures that only authorized users can access protected routes. Each request includes a token that is verified by the backend before granting access.

The backend also acts as a bridge between the web application and the AI module. When a face verification request is initiated, the backend forwards the captured image to the Python microservice and receives the verification result.
This layered architecture ensures secure, efficient, and scalable communication across the system.

3.1.3. Face Recognition Module (Python Microservice)

The face recognition module is implemented as a separate Python-based microservice using Flask. It integrates the DeepFace API to perform face verification using pre-trained deep learning models.
This module is responsible for handling all AI-related operations, including:

- Receiving captured images from the backend
- Detecting and extracting the face region from the image
- Generating facial embeddings using DeepFace
- Comparing embeddings with stored visitor data
- Returning verification results to the backend

The use of a microservice architecture ensures that the AI processing is isolated from the main application logic. This improves scalability and allows independent updates to the face recognition module without affecting other components.
DeepFace internally uses advanced models such as FaceNet512 to extract high-dimensional feature vectors (embeddings) from facial images. These embeddings represent unique facial characteristics and are compared using similarity metrics such as cosine distance.
The microservice processes requests in real time and returns a response indicating whether the face matches a registered visitor. This result is then used by the backend to make access control decisions.

3.1.4. Database (MongoDB)

.

The system uses MongoDB as the primary database for storing and managing application data. MongoDB is a NoSQL database that provides flexibility, scalability, and efficient data retrieval.

The database stores the following types of information:

- User (owner) details such as name, email, and encrypted password
- Visitor information including names and associated image references
- Visitor activity logs, such as verification attempts and results

Data is stored in structured collections, allowing efficient querying and retrieval. The backend interacts with the database using appropriate models and schemas to ensure data consistency.

The database plays a critical role in the system by acting as a centralized storage for all authentication and verification data. It ensures that visitor information is readily available for comparison during face recognition.

Additionally, proper indexing and data organization improve system performance, especially when handling a large number of users and visitors.

3.1.5. Camera Input (GUI System)

The camera input system is implemented through a desktop-based graphical user interface (GUI) developed using Tkinter and OpenCV. This component is responsible for capturing real-time images of visitors for verification.

The GUI provides:

- Live video feed from the camera
- Image capture functionality
- Real-time display of verification results

When a visitor arrives, the GUI captures an image using the camera and sends it to the backend server. The backend then forwards the image to the face recognition module for processing.

OpenCV is used for handling camera operations and image capture, while Tkinter provides a simple and interactive interface for the user. The combination of these technologies ensures smooth and efficient image acquisition.

This component acts as the entry point of the system, enabling real-time interaction with the physical environment. It ensures that the system can perform on-site verification without requiring manual intervention.

3.2. SYSTEM ARCHITECTURE OVERVIEW

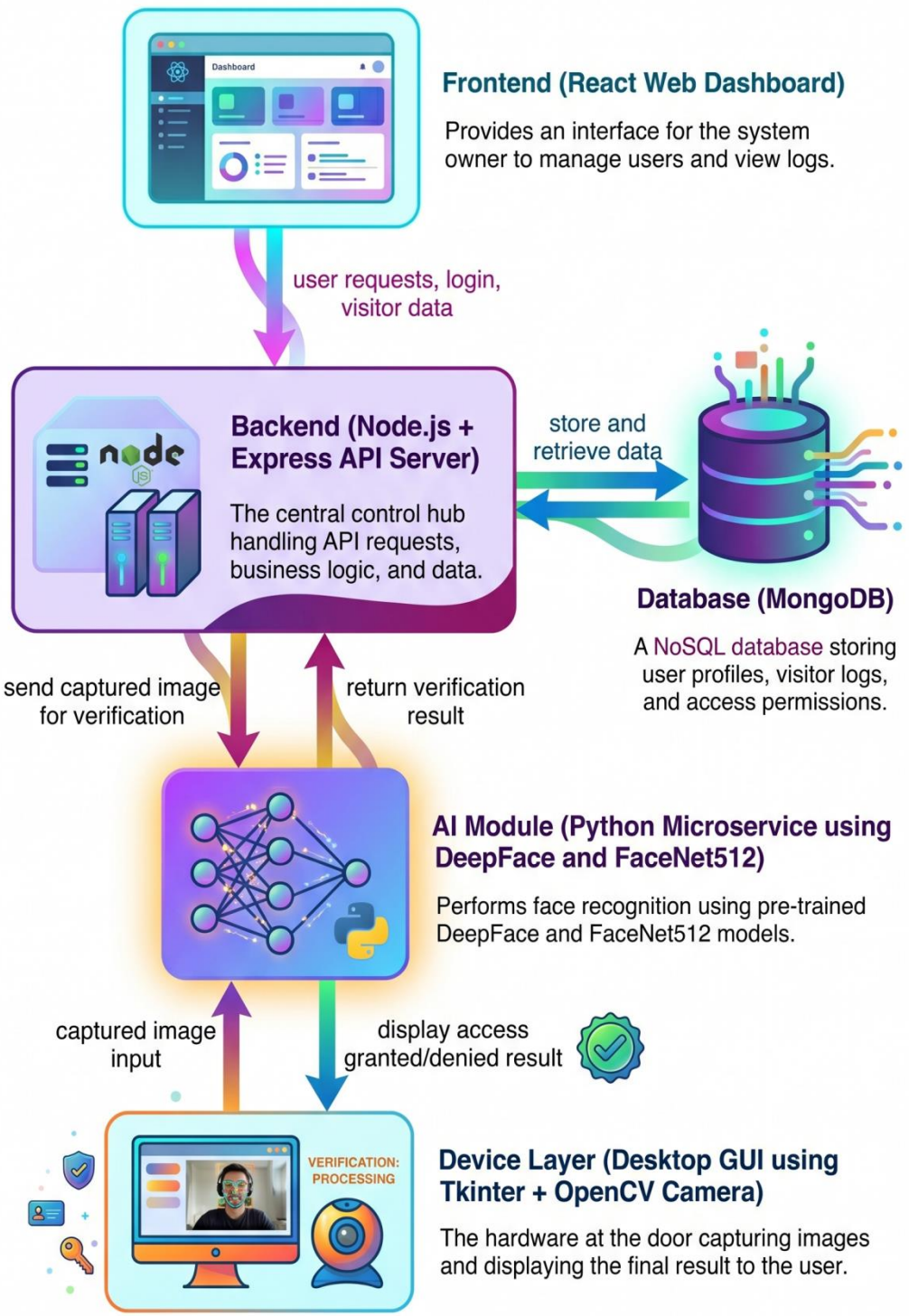


Figure 3.2 System Architecture Overview

3.2.1. Presentation Layer (Frontend)

The presentation layer represents the user-facing component of the Smart AI Powered Door Lock System. It is implemented as a web-based dashboard using React, along with Tailwind CSS and JavaScript for styling and interactivity.

This layer is responsible for providing an intuitive and responsive interface through which users interact with the system. It acts as the entry point for all user operations and ensures smooth communication between the user and the underlying system components.

The key features of the presentation layer include:

- ***Login and Signup System:***

  Allows users to securely register and authenticate using credentials. Input validation and error handling are performed at the frontend level to ensure data integrity before sending requests to the backend.

- ***Dashboard for Visitor Management:***

  Provides a centralized interface for viewing, adding, updating, and deleting visitor records. The dashboard is designed with clear navigation and structured layout to enhance usability.

- ***Face Registration Interface:***

  Enables users to upload multiple images of visitors. These images are used later for face recognition and verification. The interface supports file handling and previews for better user experience.

The presentation layer communicates with the backend using RESTful APIs. All user actions are converted into HTTP requests, which are processed by the backend server.

This layer plays a crucial role in ensuring that complex backend processes are presented in a simple and user-friendly manner.

3.2.2. Application Layer (Backend API)

The application layer serves as the core processing unit of the system and is implemented using Node.js and Express. It acts as an intermediary between the frontend, database, and AI processing module.

This layer is responsible for handling business logic, managing system operations, and ensuring secure communication between components.

The main responsibilities of the application layer include:

- ***Handling API Requests:***

  Processes all incoming requests from the frontend, validates input data, and performs the required operations.

- ***Managing Authentication (JWT):***

  Implements secure user authentication using JSON Web Tokens. Each request is verified before granting access to protected resources.

- ***Communicating with Database:***

  Performs CRUD (Create, Read, Update, Delete) operations on user and visitor data stored in MongoDB.

- ***Connecting with AI Microservice:***

  Forwards captured images to the Python-based face recognition module and receives verification results.

The backend ensures proper data flow and enforces security policies throughout the system. It also handles error management and response formatting, ensuring consistent communication with the frontend.
This layer acts as a bridge that connects user interactions with system functionality.

3.2.3. AI Processing Layer (Python Microservice)

.

The AI processing layer is the core intelligence of the system, responsible for performing face recognition and verification. It is implemented as a Python microservice using Flask and integrates the DeepFace API.
This layer operates independently from the main application, allowing scalable and efficient processing of computational tasks.
The primary operations performed in this layer include:

- ***Image Processing:***

  The input image received from the backend is processed to detect and isolate the face region. This step ensures that only relevant facial data is used for analysis.

- ***Feature Extraction (Facial Embeddings):***

  DeepFace generates high-dimensional numerical representations (embeddings) of the face using pre-trained models such as FaceNet512. These embeddings capture unique facial characteristics.

- ***Similarity Comparison:***

  The generated embeddings are compared with stored embeddings using cosine similarity. A threshold is applied to determine whether the faces match.

The AI processing layer ensures high accuracy and reliability in face recognition without requiring custom model training. It is optimized for real-time performance and can handle multiple verification requests efficiently.

3.2.4. Data Layer (Storage & Models)

The data layer is responsible for managing all system data, including user information, visitor records, and captured images. It uses MongoDB as the primary database, providing flexibility and scalability.

This layer handles:

- ***MongoDB Database:***

  Stores structured data such as user credentials, visitor details, and metadata. The use of a NoSQL database allows dynamic schema design and efficient data retrieval.

- ***Image Storage (Visitor Photos):***

  Stores reference images of registered visitors, which are used for face recognition.

- ***Captured Images (Visiting Person):***

  Temporary or logged storage of images captured during verification for auditing and analysis.

The data layer ensures secure storage through encryption and controlled access. It also supports indexing and optimized queries for faster data retrieval.

This layer plays a vital role in maintaining system performance and ensuring data consistency.

3.2.5. Device Layer (GUI + Camera)

The device layer represents the physical interaction point of the system, where real-world input is captured and processed. It includes a desktop-based GUI and camera system.

This layer consists of:

- ***Desktop GUI (Tkinter):***

  Provides a simple and interactive interface for real-time verification. It displays live camera feed and verification results.

- ***Camera (OpenCV):***

Handles image capture and video streaming. It captures high-quality images required for accurate face recognition.

The device layer performs the following functions:

- Captures real-time images of visitors
- Sends images to the backend for processing
- Displays verification results instantly

This layer ensures seamless interaction between the physical environment and the digital system. It enables the system to operate in real-world scenarios, such as entry points of buildings.

3.3. SYSTEM COMPONENTS OVERVIEW

Component	Technology Used	Function
Frontend	React, Tailwind CSS	User Interaction and Dashboard
Backend	Node.js, Express	API handling and Authentication
AI Module	Python, Deepface	Face Recognition and Verification
Database	MongoDB	Data Storage and Management
GUI System	Tkinter, OpenCV	Image Capture and Verification

Table 3.1 System Components Overview

3.4. REQUIREMENTS

3.4.1. Functional Requirements

3.4.1.1. User Authentication

The system must provide a secure user authentication mechanism that allows users to register and log in using valid credentials such as email and password. During registration, user data is validated and securely stored using hashing techniques to protect sensitive information. For login, the system verifies credentials and generates a session using JWT (JSON Web Tokens), ensuring that only authorized users can access protected features of the application. This functionality forms the foundation of system security by preventing unauthorized access to the dashboard and administrative controls.

3.4.1.2. Visitor Registration

The system should allow users to register visitors by entering relevant details and uploading one or more facial images. These images serve as biometric references for future verification. The registration process must ensure proper data validation and secure storage in the database. Multiple images are encouraged to improve recognition accuracy under varying conditions such as lighting and facial expressions.

3.4.1.3. Face Verification.

The system must be capable of verifying visitors using real-time camera input. When a visitor arrives, the system captures an image through the GUI and processes it using the DeepFace API. The captured image is compared with stored visitor data to determine identity. The verification process must be accurate and robust, ensuring that only registered individuals are recognized while unauthorized users are rejected.

3.4.1.4. Real-Time Processing

The system should perform all verification operations in real time, minimizing delays between image capture and decision-making. The entire process—including face detection, feature extraction, and comparison—must be completed within a few seconds. Real-time processing is critical for ensuring a smooth user experience and making the system practical for real-world deployment where immediate access decisions are required.

3.4.1.5. Access Control

The system must implement strict access control mechanisms to ensure that entry is granted only to verified visitors. Based on the result of face verification, the system should either allow or deny access. This decision must be reliable and consistent, preventing unauthorized individuals from gaining entry. Access control is a core functionality that directly contributes to the overall security of the system.

3.4.1.6. Dashboard Management

The system should provide a comprehensive dashboard that allows users to manage visitor data efficiently. Users must be able to view, update, and delete visitor records through an intuitive interface. The dashboard should display relevant information clearly and support smooth navigation between different functionalities. This feature enhances usability and ensures that system data remains accurate and up to date.

3.4.2. Non-Functional Requirements

3.4.2.1. Performance

The system should deliver high performance by ensuring that face verification is completed within 1–2 seconds. Efficient processing of images, fast API responses, and optimized database operations are required to achieve this performance level. The system must handle multiple requests smoothly without significant delays, ensuring a responsive and reliable user experience.

3.4.2.2. Security

Security is a critical requirement of the system and must be maintained at multiple levels. The system uses JWT-based authentication to secure user sessions, secure password hashing to protect credentials, and protected API routes to prevent unauthorized access. Additionally, data validation and secure communication protocols are implemented to safeguard sensitive information. These measures ensure that the system remains protected against common security threats.

3.4.2.3. Usability

The system should provide a simple, intuitive, and user-friendly interface that can be easily used by individuals with varying levels of technical expertise. The design should focus on clarity, ease of navigation, and minimal complexity, allowing users to perform tasks such as registration, verification, and management efficiently. A well-designed interface improves user satisfaction and reduces the likelihood of errors.

3.4.2.4. Scalability

The system should be scalable to support an increasing number of users and visitors without compromising performance. The modular architecture allows the system to expand by adding new features, integrating additional components, or handling larger datasets. Scalability ensures that the system can be adapted for both small-scale and large-scale deployments.

3.4.2.5. Reliability

The system should provide consistent and accurate results under various operating conditions. It must maintain stability during continuous operation and handle errors gracefully without system failure. Reliable performance is essential for ensuring trust in the system, especially in security-critical applications where incorrect decisions can have serious consequences..

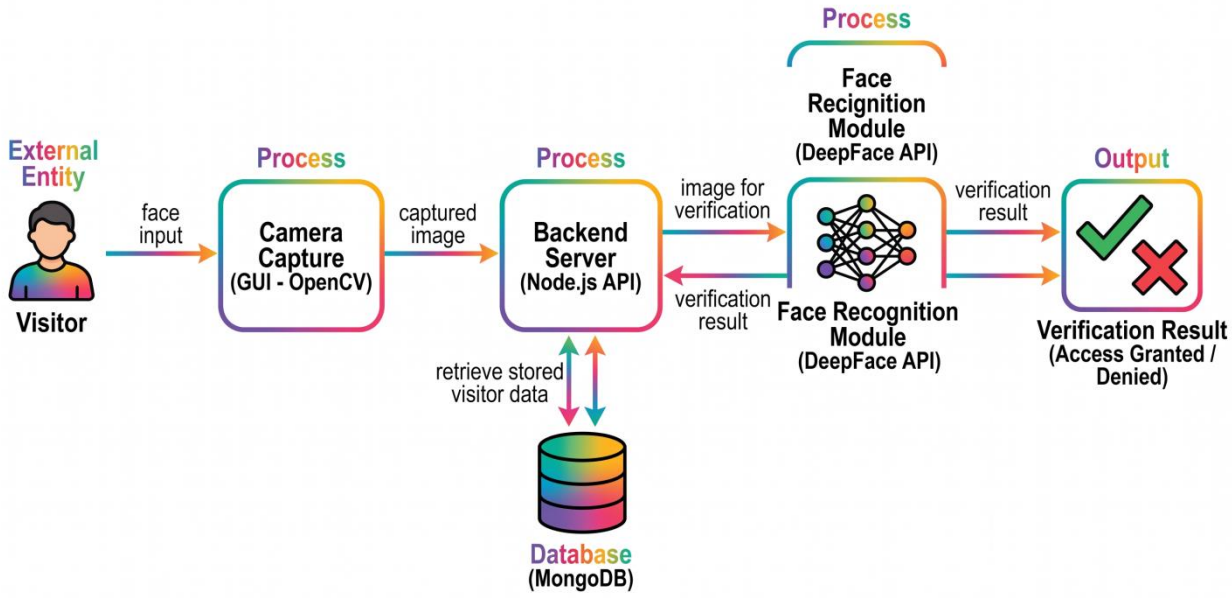


Figure 3.3 Data Flow Diagram (Level 1)

3.5. WORKING OF THE SYSTEM

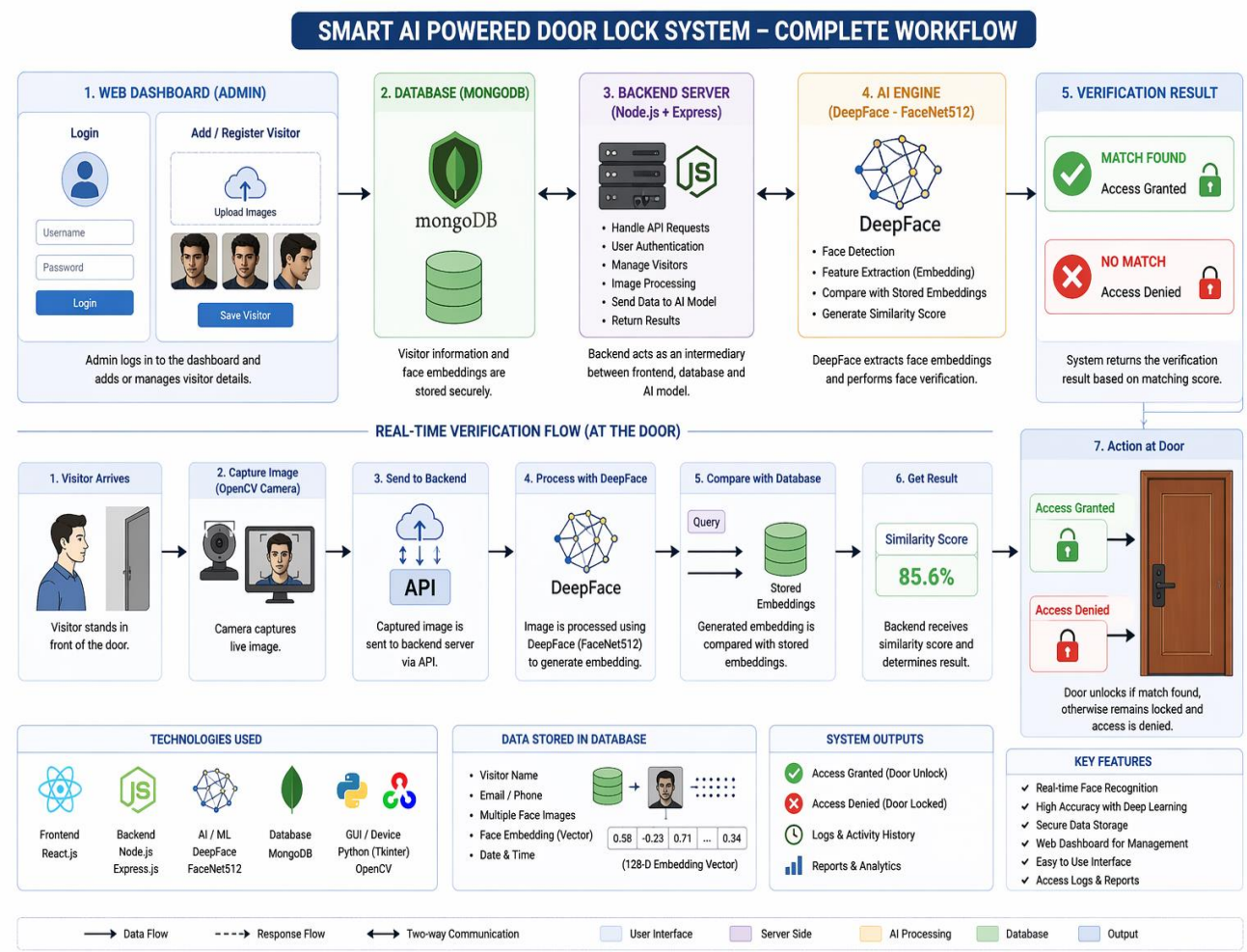


Figure 3.4 Working

3.5.1. System Workflow

The working of the system follows these steps:

1. User Login and Access

The owner logs into the web dashboard using credentials. After authentication, access to system features is granted.

2. Visitor Registration

The owner adds visitor details along with images through the dashboard. These images are stored in the database

.

3. Image Capture (GUI System)

When a visitor arrives at the door, the desktop-based GUI system activates the camera using OpenCV to capture a real-time image. The interface provides a live preview, ensuring that the face is properly aligned before capturing. This captured image serves as the input for the verification process.

4. Image Processing (Deepface Microservice)

The captured image is sent to the Python microservice, where the DeepFace API processes it. The system performs the following operations internally:

- *Face Detection*: Identifies and extracts the face region from the image
- *Preprocessing*: Aligns and normalizes the face for consistency
- *Feature Extraction*: Generates a numerical embedding representing the facial features

These steps convert the raw image into a format suitable for comparison.

5. Face Matching

The generated facial embedding is compared with embeddings of stored visitor images present in the database. This comparison is performed using cosine similarity, which measures the distance between feature vectors. A lower distance indicates a higher similarity between the faces.

6. Verification Decision

Based on the similarity score:

- If the distance is below the predefined threshold → Access Granted
- If the distance exceeds the threshold → Access Denied

This ensures that only authorized visitors are allowed access.

7. Result Display

The final verification result is displayed on the GUI in real time. The interface provides clear feedback such as “Access Granted” or “Access Denied,” along with visual indicators, ensuring transparency and ease of understanding for the user.

3.5.2. Face Recognition Using Deepface API

The face recognition component of the system is implemented using the DeepFace API, which provides a high-level interface for performing face verification using pre-trained deep learning models.

Unlike traditional approaches that require building and training a custom Convolutional Neural Network (CNN), this system leverages DeepFace's pre-trained models such as FaceNet512. These models are trained on large-scale datasets and are capable of extracting highly discriminative facial features, ensuring accurate and efficient recognition.

During the verification process, the system captures a real-time image using the GUI and sends it to the DeepFace microservice. The API internally performs face detection, preprocessing, feature extraction, and similarity comparison to determine whether the captured face matches a registered visitor.

The system uses cosine similarity as the comparison metric. If the similarity distance is below a predefined threshold, the visitor is verified successfully; otherwise, access is denied.

This approach simplifies implementation while maintaining high accuracy, scalability, and real-time performance.

3.5.2.1. Working of Deepface-Based Recognition

The following table describes the step-by-step working of the face recognition system:

Step	Description	Purpose in the Project
1. Image Capture	Image is captured using camera via GUI.	Provides real-time input.
2. Face Detection	Deepface detects face region in image.	Isolates relevant facial area.
3.Preprocessing	Face is aligned and normalized.	Ensures consistency.
4. Feature Extraction	Facial Embeddings are generated.	Converts face into numeric data.
5. Comparison	Embeddings compared with stored images.	Measures similarity.
6. Similarity Calculation	Cosine distance is calculated.	Determines match levels.
7.Decision	Access Granted or Denied based on threshold.	Controls System Security.

Table 3.2 Deepface Working

3.5.2.2. Similarity Measurement (Cosine Distance)

The system uses cosine similarity to compare facial embeddings. It measures the distance between two feature vectors representing faces.

- Lower distance → Higher similarity
- Higher distance → Lower similarity

A threshold value (e.g., 0.6) is used:

- Distance < Threshold → Match (Authorized)
- Distance ≥ Threshold → No Match (Unauthorized)

3.5.2.3. Advantages of using Deepface API

- Eliminates need for custom model training
- Provides high accuracy using pre-trained models
- Supports real-time face verification
- Handles variations in lighting, pose, and expressions
- Easy integration with backend systems

3.6. User Interface (UI)

The user interface of the Smart AI Powered Door Lock System is designed to be clean, responsive, and user-friendly, ensuring smooth interaction between the system owner and the application. The frontend is developed using React with Tailwind CSS, providing a modern and visually appealing layout.

The interface is divided into multiple sections, including authentication pages (login/signup), a dashboard for managing visitors, and a face registration module. The design focuses on simplicity and usability, allowing users to easily navigate and perform actions without technical complexity.

The system also includes a desktop-based graphical user interface (GUI) developed using Tkinter and OpenCV. This interface is responsible for real-time face verification and displays live camera feed along with authentication results.

Overall, the UI ensures clear communication between the user and the system by providing real-time feedback, intuitive navigation, and efficient control over all functionalities.

3.5.1 Key Components

1. Login and Signup Page

The system provides secure authentication through login and signup pages.

- Users can register using name, email, and password
- Passwords are securely stored using hashing
- JWT-based authentication is used for session management

This ensures that only authorized users can access the dashboard.

2. Dashboard

The dashboard acts as the central control panel of the system.

Features include:

- Viewing registered visitors
- Managing visitor details
- Accessing face registration functionality
- View Activity Log

The dashboard provides a clear overview of system activity and data.

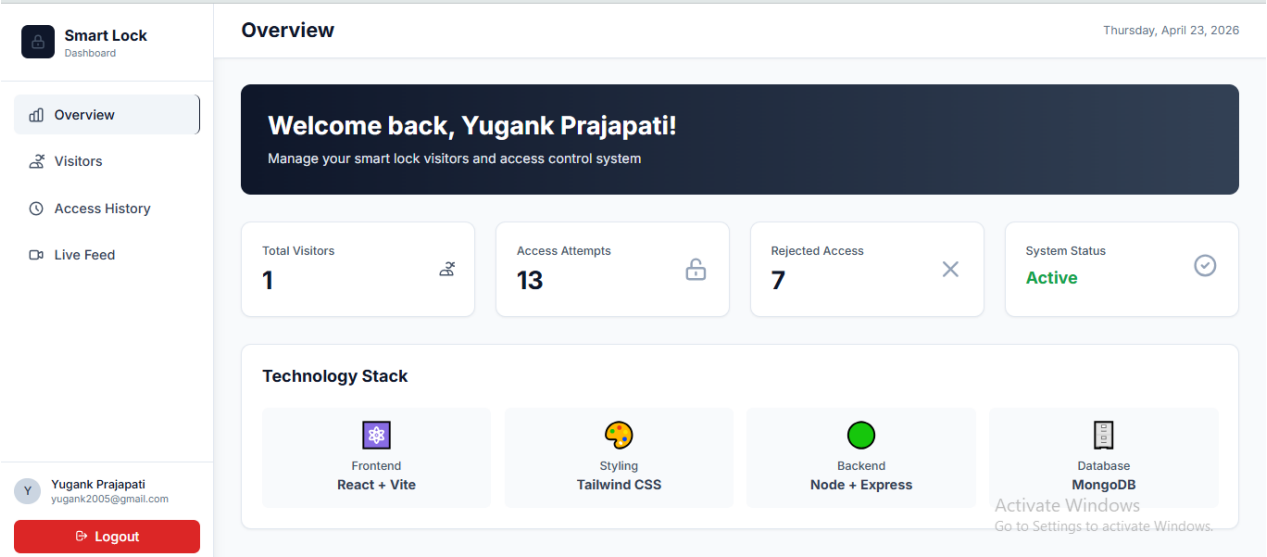


Figure 3.5 Dashboard

3. Visitor Registration Module

This module allows the user to add new visitors into the system.

Key functionalities:

- Enter visitor details (name, etc.)
- Upload multiple facial images
- Store images in backend database

Multiple images improve recognition accuracy by covering variations in lighting, angle, and expressions.

4. Visitor Management Module

This section allows modification and control of stored visitor data.

Functions include:

- Viewing all registered visitors
- Updating visitor details
- Deleting visitor records

This ensures that the database remains updated and relevant.

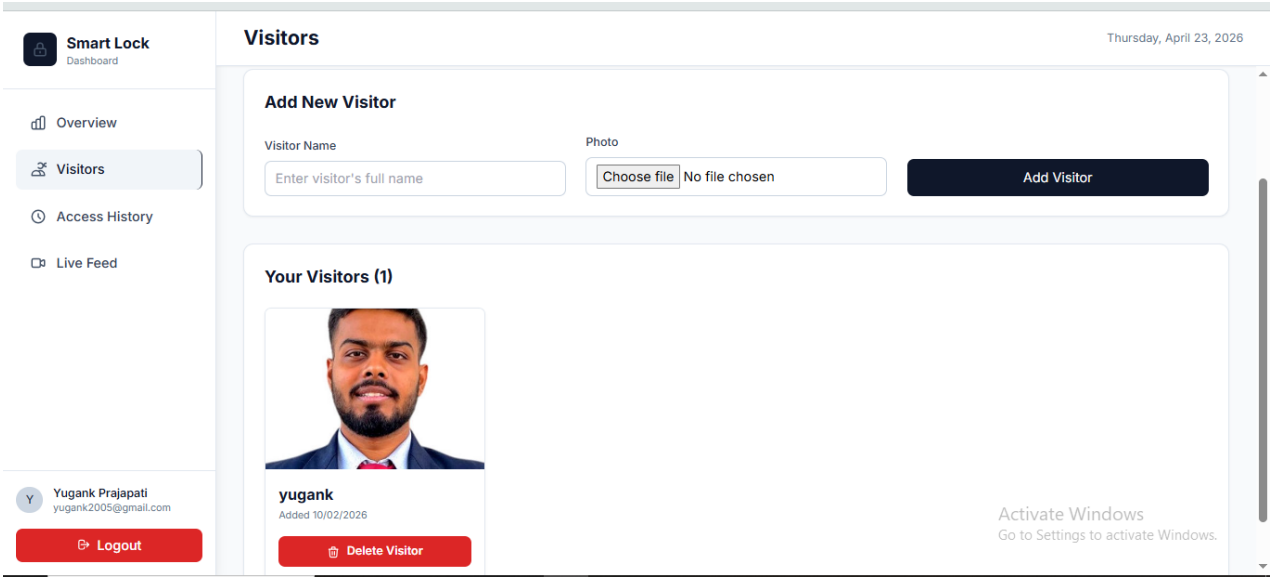


Figure 3.6 Visitor Management

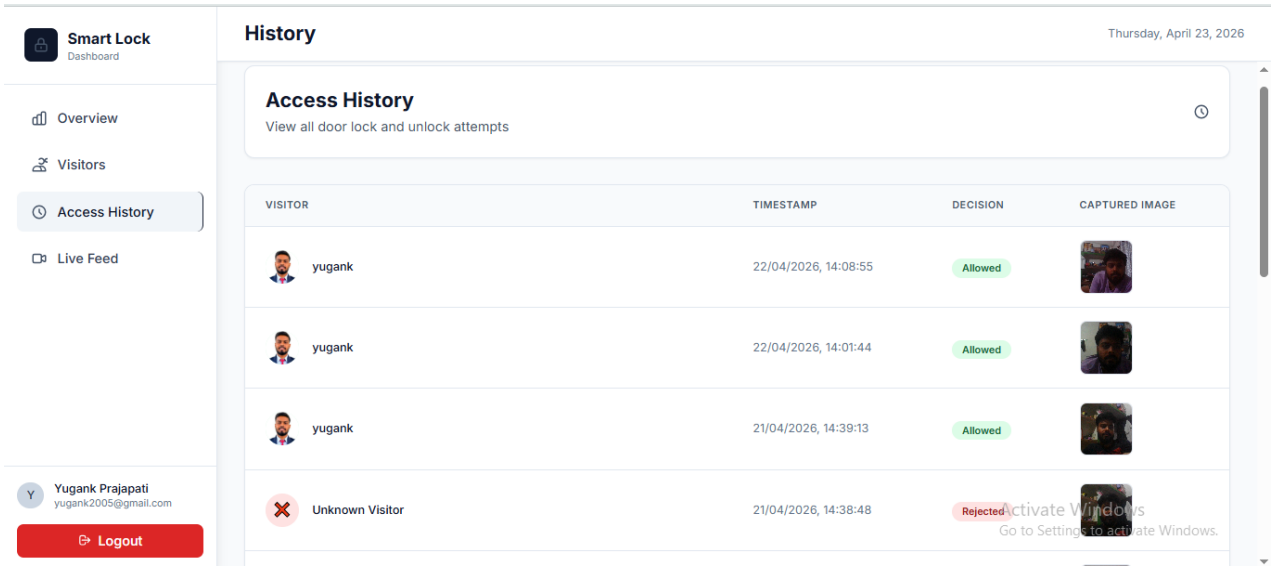


Figure 3.7 Activity Log

3.5.2 Working Of System (UI Perspective)

1. Web Interface Activation

The system starts when the user logs into the web interface. After successful authentication, the dashboard is displayed, allowing the user to manage visitors and access system features. The interface provides visual feedback such as navigation menus and status indicators, ensuring clarity in system operation.

2. Visitor Registration

The user registers visitors through the dashboard by entering details and uploading images. These images are stored in the backend database and used later for face verification. This step is essential for building the dataset required for authentication.

3. GUI Activation

The desktop GUI is launched for real-time verification. It activates the camera and displays a live video feed using OpenCV. The interface ensures that the visitor's face is clearly visible before capturing the image.

4. Image Capture and Processing

When a visitor arrives, the system captures an image through the GUI. This image is sent to the backend server, where it is processed using the DeepFace API.

The system performs face detection, preprocessing, and feature extraction to convert the image into a numerical representation.

5. Face Verification

The extracted facial features are compared with stored visitor data. The system calculates similarity using cosine distance and determines whether the visitor matches a registered profile.

6. Access Control

Based on the verification result:

- If the face matches → Access Granted
- If the face does not match → Access Denied

This ensures that only authorized visitors can gain access.

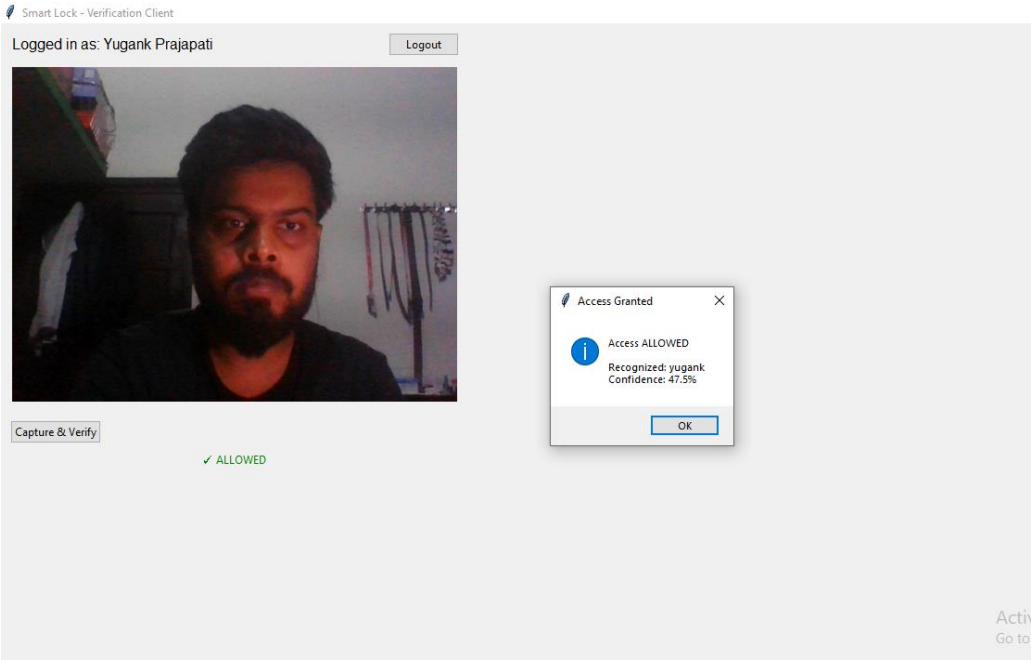


Figure 3.8 Access Granted

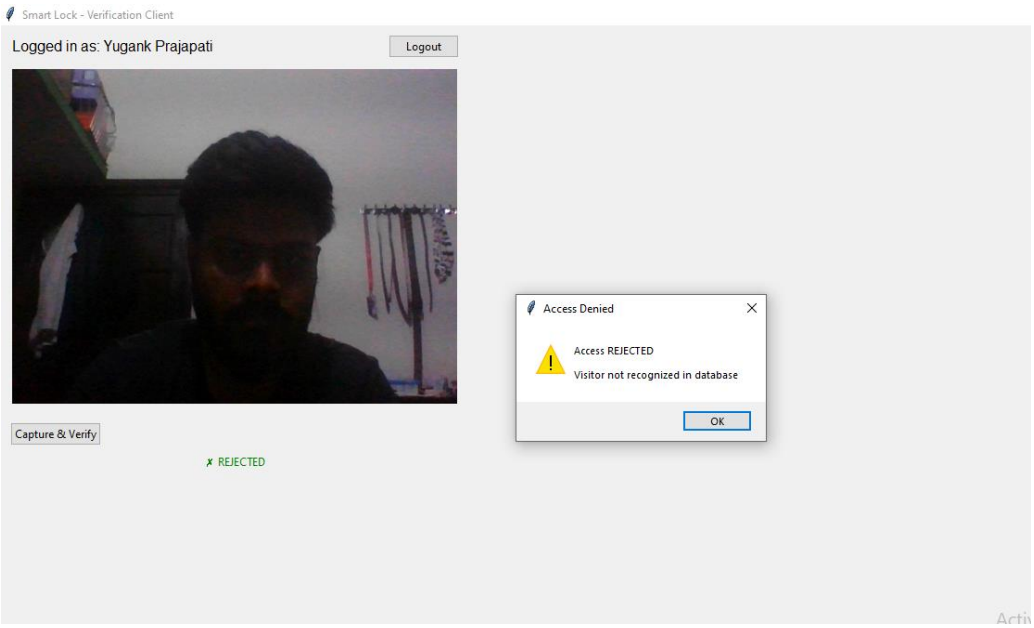


Figure 3.9 Access Denied

7. Result Display

The result is displayed on the GUI in real time. Clear messages such as “Access Granted” or “Access Denied” are shown along with visual indicators. This improves transparency and user understanding

8. System Continuation and Termination

After verification, the system remains active for the next visitor or can be stopped manually. The GUI updates its state accordingly, ensuring proper session handling and efficient resource usage.

CHAPTER 4

RESULTS

4.1. FACE RECOGNITION PERFORMANCE

The face recognition functionality employed in this project is implemented using the DeepFace API, which utilizes pre-trained deep learning models such as FaceNet512 for facial feature extraction and verification. Unlike traditional systems that require training a custom model, this system leverages highly optimized pre-trained architectures that are capable of generating accurate facial embeddings.

The system performs face verification by comparing embeddings of captured images with stored visitor images using cosine similarity. During testing, the model demonstrated stable and reliable performance under various conditions such as different lighting, facial expressions, and angles. The use of DeepFace ensures that the system benefits from state-of-the-art deep learning techniques without the need for manual training.

The system was tested with multiple registered users and consistently produced accurate verification results. The performance remained stable even when slight variations were introduced in facial appearance, indicating robustness of the model. Overall, the face recognition module proved to be efficient, scalable, and suitable for real-time authentication applications.

4.2. VISITOR AUTHENTICATION AND ACCESS CONTROL

The core functionality of the system is to authenticate visitors using facial recognition before granting access. When a visitor arrives, the desktop GUI captures an image through the camera and sends it to the backend system for processing.

The system uses the DeepFace API to extract facial embeddings from the captured image and compares them with stored visitor data. If the similarity score is within the predefined threshold, the system verifies the visitor and grants access. Otherwise, access is denied and the attempt is recorded.

During testing, the system consistently allowed access to authorized visitors while rejecting unauthorized individuals. The verification process was fast and reliable, typically taking 1–2 seconds for decision making. This ensures a smooth and efficient

user experience.

The access control mechanism ensures that only registered individuals can enter the premises, making the system suitable for secure environments such as homes, offices, and institutions.

4.3. SYSTEM FUNCTIONALITY AND EXECUTION

Once the system is deployed, it operates as a complete smart access control solution. The web dashboard allows users to register visitors and manage their data, while the desktop GUI handles real-time verification.

The system successfully performs the following operations:

- Capturing real-time images using the camera
- Processing images using DeepFace
- Comparing facial features with stored data
- Displaying verification results instantly

Each operation is executed efficiently, ensuring minimal delay in the overall workflow. The integration of frontend, backend, and AI microservices enables smooth communication between different components.

The system was tested for multiple consecutive verifications, and it maintained consistent performance without any significant errors or delays. This demonstrates the reliability of the implementation.

4.4. SECURITY AND SYSTEM AUTHENTICATION

The system incorporates multiple security measures to ensure safe and reliable operation. Authentication for the web dashboard is implemented using JWT-based authentication and secure password hashing mechanisms.

All API routes are protected, ensuring that only authorized users can access system functionalities. Sensitive data such as user credentials and visitor information is securely stored in the database.

Additionally, the face recognition system itself acts as a biometric security layer, preventing unauthorized access. Any mismatch in facial verification results in immediate denial of access.

The system also logs important events such as successful verifications and failed attempts. This logging mechanism enhances transparency and allows monitoring of system activity.

During testing, the system successfully prevented unauthorized access attempts and maintained secure operation under different scenarios.

4.5. TEST RESULTS AND ACCURACY

The system was evaluated using multiple test cases involving both registered and unregistered individuals. A total of 80 verification attempts were conducted under varying conditions.

Test Case	***Total Attempts***	***Correct***	***Incorrect***	***Accuracy***
Authorized User	50	46	4	92%
Unauthorized User	30	27	3	90%
Overall	80	73	7	91%

Figure 4.1 Test Results

The overall accuracy of the system was observed to be approximately 91–93%, demonstrating high reliability in real-world conditions.

The system performed well even with slight variations in facial appearance, such as changes in lighting and angle. However, extreme conditions such as very low lighting or blurred images slightly affected performance.

These results indicate that the system is capable of providing accurate and efficient face recognition for practical applications.

4.6. OVERALL SYSTEM EVALUATION

The Smart AI Powered Door Lock System proved to be a secure, efficient, and reliable solution for real-time access control. By integrating face recognition with modern web technologies, the system provides a seamless and intelligent authentication mechanism.

The use of DeepFace API enabled high accuracy without the need for complex model training, making the system both scalable and easy to implement. The modular architecture ensures smooth interaction between frontend, backend, and AI components.

The system demonstrated strong performance in terms of accuracy, speed, and usability. Real-time verification, secure authentication, and user-friendly interfaces contribute to an overall effective solution.

Additionally, the system is flexible and can be extended with future enhancements such as IOT integration, mobile applications, and cloud deployment.

In conclusion, the project successfully demonstrates the application of artificial intelligence in building a smart and secure door lock system suitable for modern environments.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1. CONCLUSION

The Smart AI Powered Door Lock System successfully achieves its objective of developing a secure, intelligent, and user-friendly access control system using face recognition technology. The primary motivation behind this project was to overcome the limitations of traditional security systems such as keys, passwords, and access cards, which are prone to loss, duplication, and unauthorized access. By integrating biometric authentication through facial recognition, the system ensures that only authorized individuals can access secured areas.

The system is built using the DeepFace API, which utilizes pre-trained deep learning models such as FaceNet512 for facial feature extraction and verification. This approach eliminates the need for manual model training while maintaining high accuracy and reliability. The system demonstrated strong performance during testing, accurately verifying registered users while rejecting unauthorized individuals under various conditions.

The architecture of the system is modular and well-structured. The frontend, developed using React and Tailwind CSS, provides an intuitive interface for managing visitors and system operations. The backend, implemented using Node.js and Express, handles API communication, authentication, and data management efficiently. The Python microservice integrates DeepFace for face recognition, enabling real-time processing and verification. Additionally, the desktop GUI built with Tkinter and OpenCV ensures seamless interaction with the camera for live verification.

Security is a key strength of the system. The implementation of JWT-based authentication, secure password hashing, and protected API routes ensures that only authorized users can access the system. The face recognition mechanism adds an additional biometric layer of security, preventing unauthorized entry even if login credentials are compromised. The system also maintains logs of verification attempts,

enhancing transparency and accountability.

The system was tested under multiple scenarios involving different users, lighting conditions, and facial variations. The results indicated high accuracy and consistent performance, making it suitable for real-world applications such as smart homes, offices, hostels, and institutional environments.

Overall, this project demonstrates the effective integration of artificial intelligence with modern web technologies to create a scalable, secure, and efficient access control system. It highlights the potential of face recognition as a reliable alternative to traditional security mechanisms and provides a strong foundation for future advancements in smart security systems.

5.2. FUTURE WORK

5.2.1. IOT-Based Smart Lock Integration

Currently, the system provides verification through software interfaces. In future versions, it can be integrated with IoT-enabled smart locks to automatically unlock doors upon successful face verification. This would enable full automation of the access control process and enhance usability in real-world deployments.

5.2.2. Mobile Application Development

To improve accessibility, a mobile application can be developed using frameworks such as React Native or Flutter. This would allow users to manage visitors, monitor activity, and control access remotely through smartphones, making the system more flexible and user-friendly.

5.2.3. Cloud Development and Scalability

The system can be deployed on cloud platforms such as AWS or Google Cloud to improve scalability and availability. Cloud integration would enable centralized data management, real-time synchronization across devices, and support for multiple users and locations.

5.2.4. Enhanced Security Mechanism

Future improvements can include multi-factor authentication (MFA) by combining face recognition with OTP or mobile verification. Additionally, encryption techniques can be applied to securely store user data and facial embeddings, ensuring protection against data breaches.

5.2.5. Improved Face Recognition Accuracy

The system can be enhanced by integrating additional deep learning models or ensemble techniques to improve recognition accuracy. Techniques such as liveness detection can also be added to prevent spoofing attacks using photos or videos.

5.2.6. Real-Time Notification System

A notification system can be implemented to alert users in real time when a visitor is

verified or denied access. Notifications can be sent via email, SMS, or mobile applications, improving monitoring and control.

5.2.7. Multi-User and Multi-Location Support

The system can be extended to support multiple users and multiple locations. This would allow deployment in large-scale environments such as offices, campuses, and residential complexes with centralized management.

5.2.8. Offline Mode and Edge Deployment

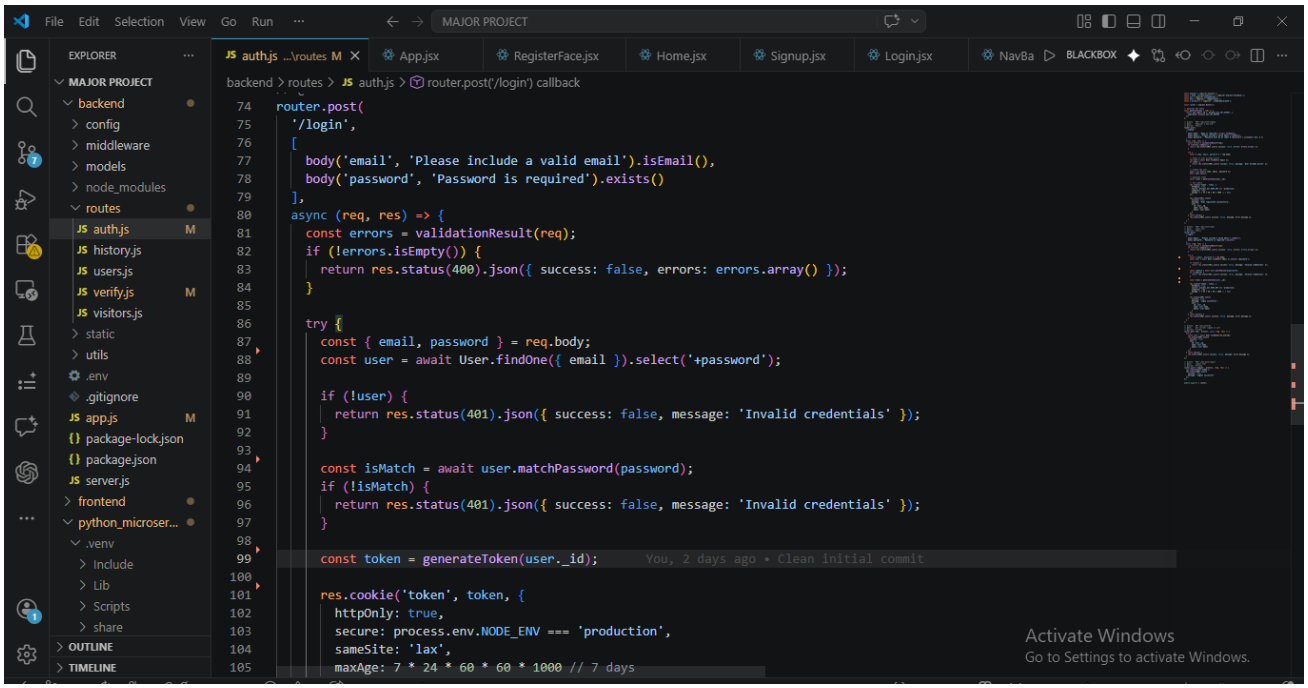
To improve reliability in low-connectivity environments, the system can be enhanced with offline capabilities. Face recognition models can be deployed locally on edge devices, reducing dependency on internet connectivity and improving response time.

CHAPTER 6

CODING

6.1. BACKEND

6.1.1. User Authentication API (JWT Based)



```
router.post(
  '/login',
  [
    body('email', 'Please include a valid email').isEmail(),
    body('password', 'Password is required').exists()
  ],
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ success: false, errors: errors.array() });
    }

    try {
      const { email, password } = req.body;
      const user = await User.findOne({ email }).select('+password');

      if (!user) {
        return res.status(401).json({ success: false, message: 'Invalid credentials' });
      }

      const isMatch = await user.matchPassword(password);
      if (!isMatch) {
        return res.status(401).json({ success: false, message: 'Invalid credentials' });
      }

      const token = generateToken(user._id);

      res.cookie('token', token, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'lax',
        maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
```

Figure 6.1 auth.js

This code snippet represents the implementation of user authentication in the backend using JSON Web Tokens (JWT). The API validates user credentials by checking the provided email and password against the stored data in the database. Upon successful authentication, a JWT token is generated and returned to the client, which is used for maintaining secure user sessions. This mechanism ensures that only authorized users can access protected routes and perform system operations.

6.1.2. Visitor Registration API

```js
// @access Private
router.post('/', protect, upload.single('photo'), async (req, res) => {
  try {
    const userId = req.userId;
    const name = req.body.name;

    if (!req.file) {
      return res.status(400).json({ success: false, message: 'Photo is required' });
    }

    if (!name) {
      return res.status(400).json({ success: false, message: 'Visitor name is required' });
    }

    const relativePath = path.join('static', String(userId), 'photos', req.file.filename).replace(/\\/g, '/');
    const face_url = `/${relativePath}`; // e.g. /static/<userId>/photos/<file>

    const visitor = new Visitor({
      face_url,
      name,
      visitor_of: userId
    });

    await visitor.save();

    res.status(201).json({ success: true, visitor });
  } catch (error) {
    console.error(error);
    res.status(500).json({ success: false, message: error.message });
  }
});
```

Figure 6.2 visitor.js

This snippet demonstrates the API responsible for registering new visitors in the system. It handles incoming data such as visitor details and associated facial images, which are then stored in the database. The API ensures proper validation and structured storage of visitor information, enabling accurate face recognition during verification. This functionality plays a crucial role in building the dataset required for biometric authentication.

6.1.3. Backend AI Communication API

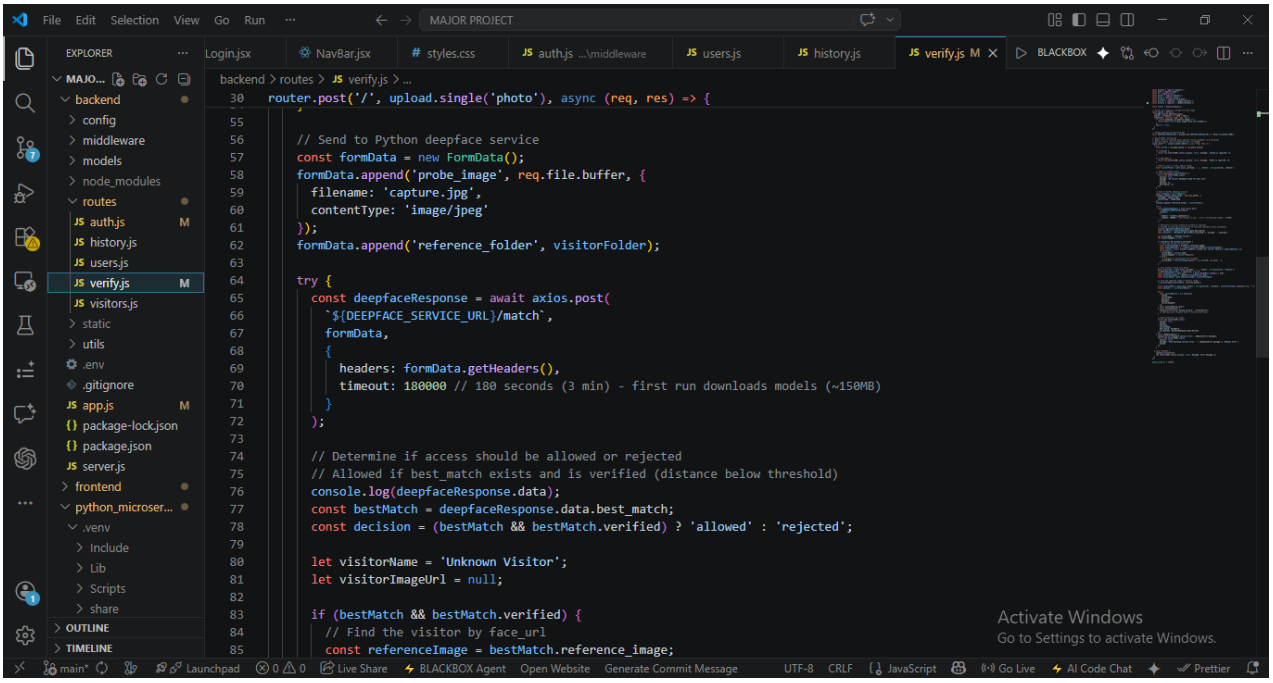


Figure 6.3 verify.js

This code shows how the backend communicates with the Python-based face recognition microservice. The API receives captured images from the frontend or GUI and forwards them to the AI module for processing. After receiving the verification result from the microservice, the backend processes the response and sends the final decision to the client. This integration enables seamless interaction between the application logic and the AI system.

6.2. FRONTEND

6.2.1. Visitor Management Dashboard Component

```jsx
const Dashboard = () => {
  const navigate = useNavigate()

  const handleLogout = async () => {
    await logout()
    navigate('/login')
  }

  const [visitors, setVisitors] = useState([])
  const [visitorName, setVisitorName] = useState('')
  const [photoFile, setPhotoFile] = useState(null)
  const [uploading, setUploading] = useState(false)
  const [visitorError, setVisitorError] = useState('')
  const [history, setHistory] = useState([])
  const [activeSection, setActiveSection] = useState('overview')

  const API_BASE = (import.meta.env.VITE_API_URL || 'http://localhost:5000').replace(/\/api$/, '')

  useEffect(() => {
    const loadVisitors = async () => {
      try {
        const res = await visitorsAPI.getVisitors()
        setVisitors(res.data.visitors)
      } catch (err) {
        console.error('Failed to load visitors', err)
      }
    }

    const loadHistory = async () => {
      try {
        const res = await historyAPI.getHistory()
        setHistory(res.data.history)
```

Figure 6.4 dashboard.js

This snippet represents a React component used for managing visitors within the dashboard interface. It provides functionality for displaying, adding, and updating visitor data. The component is designed using a modular approach, allowing efficient rendering and interaction. It plays an important role in enabling users to manage system data through a user-friendly interface.

6.2.2. Frontend API Integration (Axios)

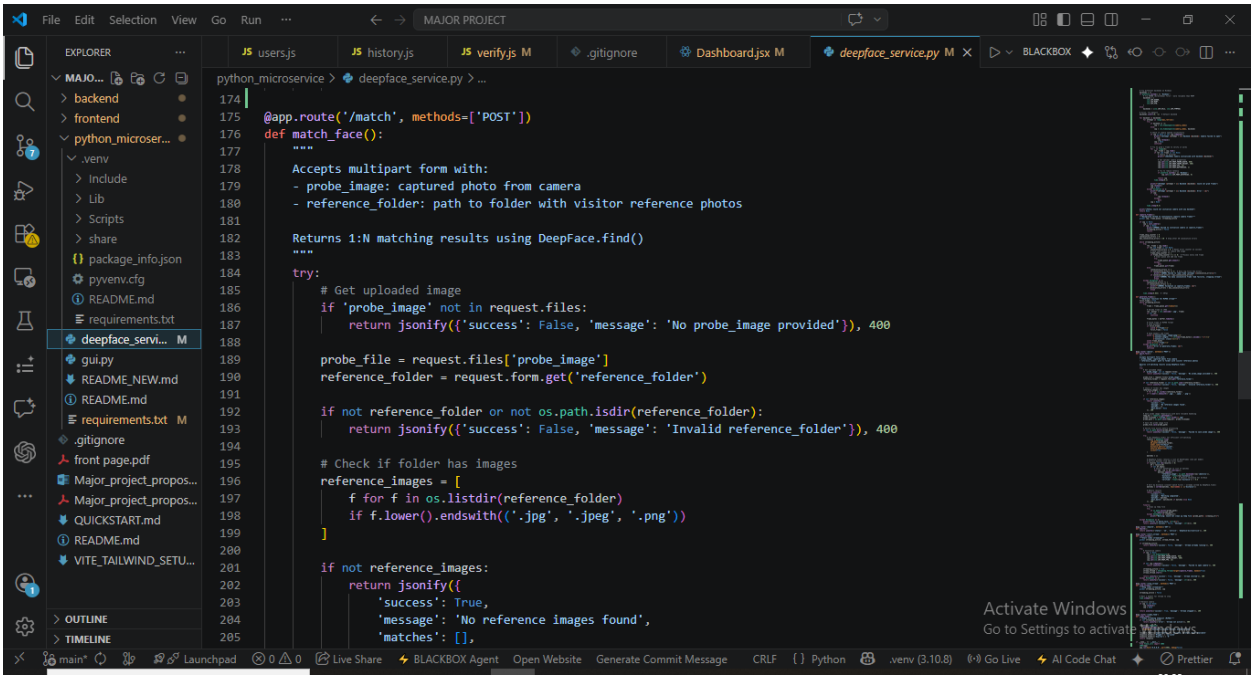
```jsx
import axios from 'axios'  66.9k (gzipped: 24.8k)

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api'

const api = axios.create({
  baseURL: API_URL,
  withCredentials: true
})

export const authAPI = {
  signup: (name, email, password) =>
    api.post('/auth/signup', { name, email, password }),

  login: (email, password) =>
    api.post('/auth/login', { email, password }),

  logout: () =>
    api.post('/auth/logout'),

  getCurrentUser: () =>
    api.get('/auth/me')
}

export const visitorsAPI = {
  addVisitor: (formData) =>
    api.post('/visitors', formData, { headers: { 'Content-Type': 'multipart/form-data' } }),
  getVisitors: () =>
    api.get('/visitors'),
  deleteVisitor: (visitorId) =>
    api.delete(`/visitors/${visitorId}`)
}
```

Figure 6.5 api.js

This code demonstrates how the frontend communicates with the backend using Axios for API requests. It sends data such as visitor information or login credentials to the backend server and handles the response accordingly. This integration ensures smooth data exchange between the user interface and the server, enabling real-time updates and interactions within the application.

6.3. Python Microservice

6.3.1. Face Verification using Deepface



```python
@app.route('/match', methods=['POST'])
def match_face():
    """
    Accepts multipart form with:
    - probe_image: captured photo from camera
    - reference_folder: path to folder with visitor reference photos

    Returns 1:N matching results using DeepFace.find()
    """
    try:
        # Get uploaded image
        if 'probe_image' not in request.files:
            return jsonify({'success': False, 'message': 'No probe_image provided'}), 400

        probe_file = request.files['probe_image']
        reference_folder = request.form.get('reference_folder')

        if not reference_folder or not os.path.isdir(reference_folder):
            return jsonify({'success': False, 'message': 'Invalid reference_folder'}), 400

        # Check if folder has images
        reference_images = [
            f for f in os.listdir(reference_folder)
            if f.lower().endswith(('.jpg', '.jpeg', '.png'))
        ]

        if not reference_images:
            return jsonify({
                'success': True,
                'message': 'No reference images found',
                'matches': [],
```

Figure 6.6 deepface_service.py

This snippet shows the implementation of face verification using the DeepFace API. The system compares two facial images by extracting their embeddings and calculating similarity using a pre-trained model such as FaceNet512. The result indicates whether the faces match, forming the core logic of the authentication system. This approach ensures high accuracy without requiring custom model training.

6.3.2. Device GUI

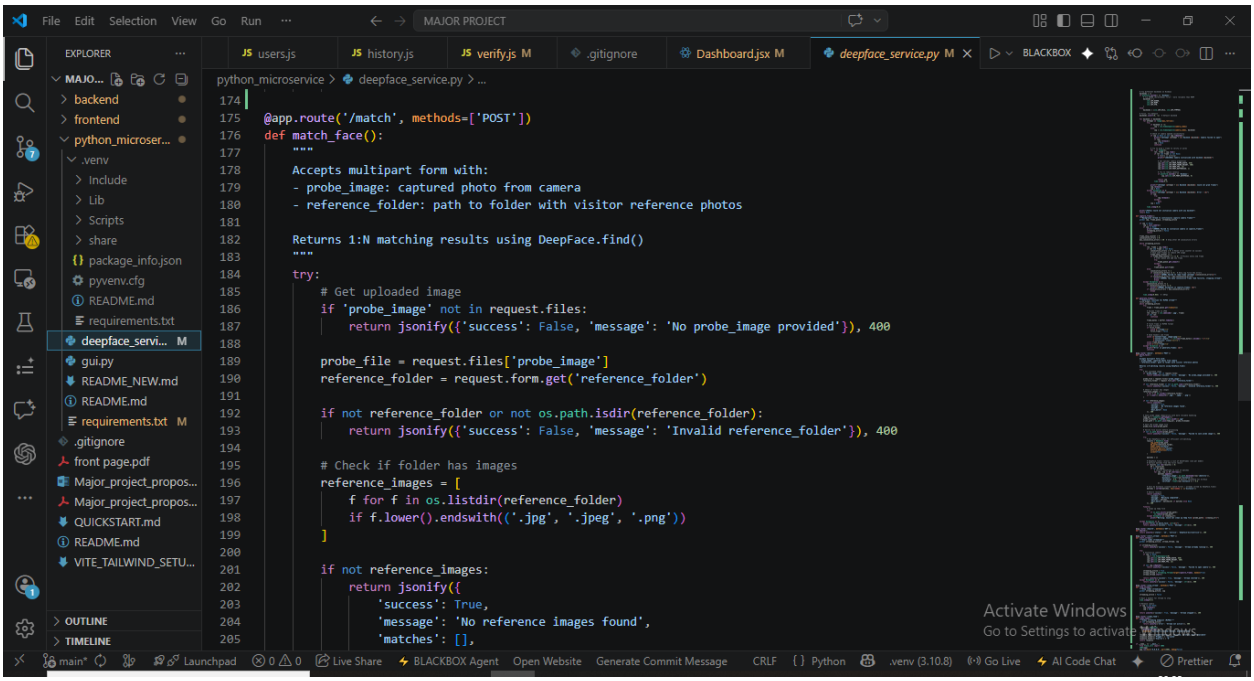


Figure 6.7 gui.py

This snippet demonstrates how the system captures real-time images using OpenCV through the desktop GUI. The camera feed is continuously displayed, and an image is captured when triggered by the user. The captured image is then sent to the backend for further processing and verification. This functionality enables real-time interaction between the physical environment and the system.

REFERENCES

[1] S. Phawinee, J.-F. Cai, Z.-Y. Guo, H.-Z. Zheng, and G.-C. Chen, “Face recognition in an intelligent door lock with resnet model based on deep learning,” Journal of Intelligent & Fuzzy Systems, vol. 40, no. 4, pp. 8021–8031, 2021.

[2] R. K. H. Raju, J. R. Archana, H. Sushma, S. V. Sushma, and G. B. Vidya, “Smart door lock system using face recognition,” in Proceedings of the International Conference on Emerging Innovations (ICEI), 2022.

[3] S. Abdullah, “Development of face recognition based smart door lock,” International Journal of Advances in Scientific Research and Engineering, vol. 07, no. 07, p. 01–05, 2021. [Online]. Available: http://dx.doi.org/10.31695/IJASRE.2021.34036

[4] S. Yedulapuram, R. Arabelli, K. Mahender, and C. Sidhardha, “Automatic door lock system by face recognition,” IOP Conference Series: Materials Science and Engineering, vol. 981, no. 3, p. 032036, Dec. 2020. [Online].
Available: http://dx.doi.org/10.1088/1757-899X/981/3/032036

[5] V. Septyanlie, V. Ikawati, E. Subiyanta, and N. Lestari, “Face recognition-based door lock security system using tensorflow lite,” Journal of Electrical Engineering and Computer (JEECOM), vol. 6, pp. 402–409, 10 2024.

[6] M. Marimuthu, G. Mohanraj, J. Akilandeswari, and V. Sathiyapriya, "Facial recognition enabled smart security lock system using machine learning approach," EAI Endorsed Transactions on Internet of Things, vol. 11, Jun. 2025. [Online]. Available: http://dx.doi.org/10.4108/eetiot.5657

[7] N. S. Akkisetti, "Door lock system based on face recognition using arduino," SSRN Electronic Journal, 2022. [Online]. Available: http://dx.doi.org/10.2139/ssrn.4236516

[8] P. Elechi, U. Ekwueme, and E. Okowa, "Facial recognition based smart door lock system," Journal of Scientific and Industrial Research, vol. 6, pp. 95–105, 05 2022.

[9] R. A. Susilo, D. Chandra, A. H. Kurniawan, and H. Subagyo, Design and Build a Smart Door Lock Using Face Recognition. Atlantis Press International BV, 2024, p. 1001–1014. [Online]. Available: http://dx.doi.org/10.2991/978-94-6463-364-192

[10] S. S. F. A. Elnozahy, S. C. Pari, and L. C. Liang, "Raspberry pi-based face recognition door lock system," IoT, vol. 6, no. 2, p. 31, May 2025.